

Computer Hardware

Input/Output: Devices and Polling

Ksawery Możdżyński

Agenda For Today



1. Meet the hardware

ESP32-C3 in Wokwi: LED, buttons, LCD



2. Talking to devices

Device registers and polling

From last time: the CPU waits

The laptop CPU from last time: **3 GHz**, one cycle $\approx$ 0.3 ns.

Waiting for		Time	Cycles
	main memory (last time)	100 ns	about 300
	a block from an SSD	50 μ s	about 150 000
	the next key you press	100 ms	about 300 000 000

Devices are **very** slow. Today: how the CPU talks to them without wasting its time.

The diagram shows the ESP32-C3-DevKitM-1 development board. It features a central ESP32-C3-MINI-1 chip. The board has two rows of pins: a top row with pins 18 (GND), 19 (GND), 0 (GND), 1 (5V), 2 (3V3), 3 (3V3), 4 (GND), 5 (RST), 6 (GND), 7 (GPIO8), 8 (GND), 9 (TX), and 10 (RX); and a bottom row with pins 11 (GND), 12 (5V), 13 (GND), 14 (GND), 15 (GND), 16 (GND), 17 (GND), and 18 (GND). The board also includes a BOOT button, an RST button, and a USB-C port.

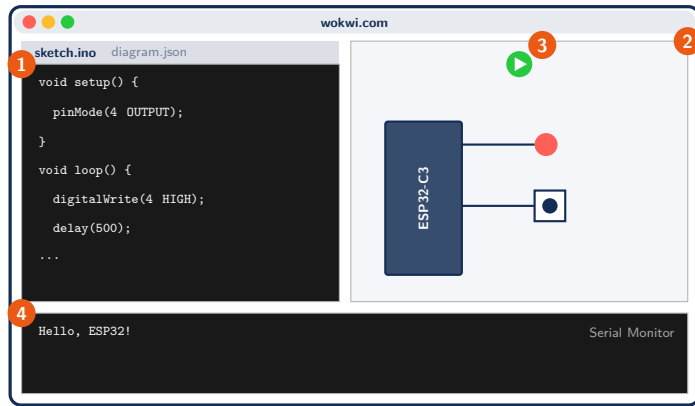
RISC-V: the same instruction set as in Ripes. One core, up to 160 MHz.

400 KB of RAM on the chip.

Pins (GPIO) for LEDs, buttons, displays and sensors.

A whole computer on one chip: CPU, memory and I/O.

Wokwi in one minute



1 Code
The program (`sketch.ino`), in Arduino C++.

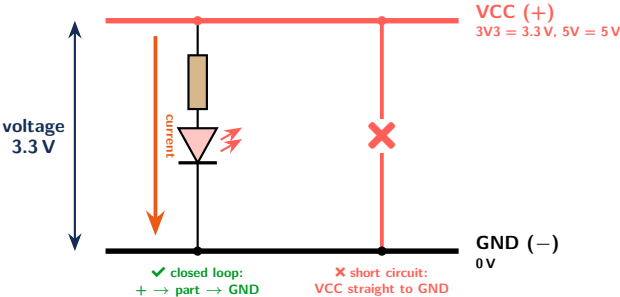
2 Diagram
The board and the parts, connected with wires.

3 Run
Compiles the program and starts the simulation.

4 Serial Monitor
Text that the program prints.

Everything runs in the browser: nothing to install, nothing to break.

Power: VCC and GND



VCC (+)

The supply: it pushes the current, like pressure in a pipe. Pins **3V3** and **5V**.

GND (-), “ground”

0 V: every voltage is measured from here. All GND pins are connected together.

A closed loop

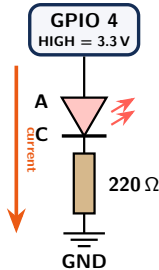
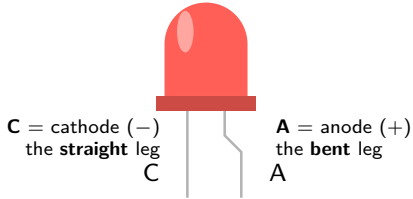
Current flows only in a loop: from + through the part back to GND.

A GPIO pin

HIGH = 3.3 V, like VCC. LOW = 0 V, like GND.

Never connect VCC straight to GND: that is a short circuit.

The LED: which leg goes where?



One way only

Current flows only from **A** (+) to **C** (-). Turned around, the LED stays dark.

How to connect

A → the GPIO pin. **C** → resistor → **GND**.

Why a resistor?

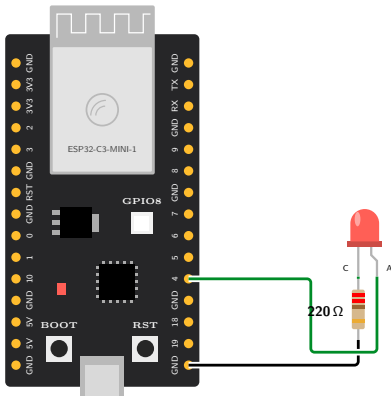
It limits the current. Without it, a real LED (and the pin) can burn.

A real LED

The **longer** leg is A. The flat edge of the case is on the C side.

GPIO 4 = HIGH: current flows **A** → **C** → resistor → GND, and the LED lights up.

Build it: the LED



1 Project

wokwi.com: **ESP32** → Starter Templates → **ESP32-C3**.

2 Parts

Click +, add an **LED** and a **Resistor** (220 Ω).

3 Wires

Click a pin, then the other pin. GPIO 4 → LED A (the bent leg). LED C → resistor → **GND**.

4 Check

Move the mouse over a pin to see its name.

Current flows from GPIO 4 through the LED and the resistor back to GND.

Task 1: blink

```
● ● ●
#define LED 4

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, HIGH); // on
  delay(500);              // wait 500 ms
  digitalWrite(LED, LOW);  // off
  delay(500);
}
```

Task

Open the project **Blink** from the course page.

Or build it as on the previous slide: template **ESP32-C3** (not ESP32, -S3, -C6 or XIAO).

1. Press **Run** and watch the LED.
2. Make the LED blink every **100 ms**.
3. Make it blink **twice** quickly, then wait **1 second**, again and again.

setup() runs once at the start, loop() runs again and again.

Solution: blink

```
void loop() {  
  for (int k = 0; k < 2; k++) {  
    digitalWrite(LED, HIGH);  
    delay(100);  
    digitalWrite(LED, LOW);  
    delay(100);  
  }  
  delay(1000);           // pause  
}
```

pinMode

Pin 4 is an **output**: the program sets it.

digitalWrite

HIGH = LED on, LOW = LED off.

delay

The CPU waits and does **nothing else**. Remember this.

GPIO: pins the program controls

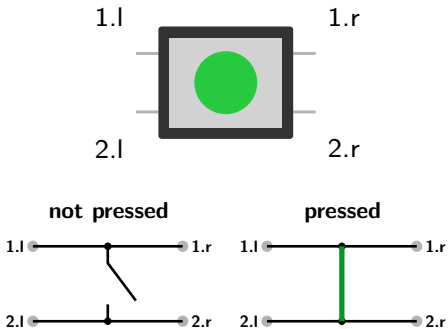


GPIO (General Purpose Input/Output): a pin that the program can **set** or **read**.

INPUT_PULLUP

The button pin reads **1** when not pressed, **0** when pressed.

The pushbutton: which legs to use?



Four legs, two pairs

1.l and 1.r are **always** connected. So are 2.l and 2.r.

Pressing

Connects pair 1 with pair 2.

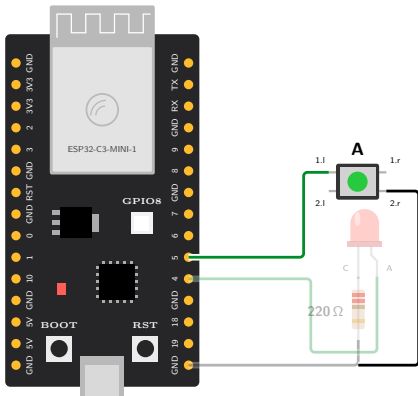
✓ Correct

One wire on a 1 leg, the other on a 2 leg. We use 1.l → GPIO 5 and 2.r → GND.

✗ Wrong

1.l and 1.r: always connected. The pin always reads 0, as if the button were always pressed.

Build it: add button A



1 Part

Keep the LED. Click +, add a **Pushbutton**.

2 Wires

GPIO 5 → button pin 1.l. Button pin 2.r → GND.

3 No bouncing

In `diagram.json`, in the button's `attrs`, add:

```
"bounce": "0"
```

No resistor for the button: `INPUT_PULLUP` gives the pin a 1 when the button is not pressed.

Task 2: a button controls the LED

```
● ● ●  
#define LED 4  
#define BTN_A 5  
  
void setup() {  
    pinMode(LED, OUTPUT);  
    pinMode(BTN_A, INPUT_PULLUP);  
}  
  
void loop() {  
    // TODO: LED on only while  
    //       the button is pressed  
}
```

Task

Open the project **Button**. Complete `loop()`: the LED is on only while the button is pressed.

`digitalRead(BTN_A)` gives HIGH or LOW.

In the simulation, click the button with the mouse and hold it.

Solution: a button controls the LED



```
void loop() {  
  if (digitalRead(BTN_A) == LOW) {    // pressed  
    digitalWrite(LED, HIGH);  
  } else {                             // not pressed  
    digitalWrite(LED, LOW);  
  }  
}
```

Why LOW?

With INPUT_PULLUP, a pressed button connects the pin to ground: it reads **0**.

How often?

loop() runs again and again, so the LED follows the button at once.

The LCD 16×2



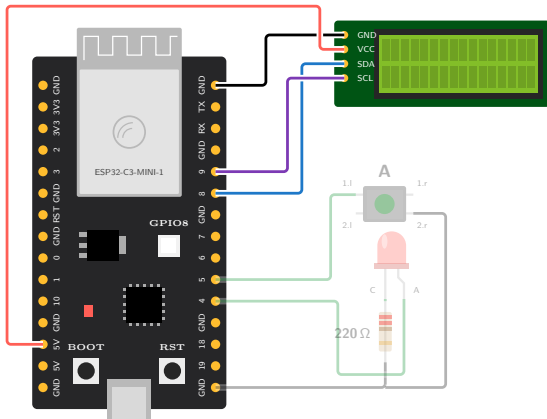
2 rows × 16 characters

```
lcd.clear();           // empty screen
lcd.setCursor(0, 1);   // column 0, row 1
lcd.print("Time: ");   // text
lcd.print(12);          // or a number
```

Only 2 wires

The LCD uses the **I2C bus**: 2 wires for all the data. How it works: after Task 3.

Build it: add the LCD



1 Part

Click +, add an **LCD 16×2 (I2C)**: it has only 4 pins.

2 Wires

SDA → GPIO 8, SCL → GPIO 9,
VCC → 5V, GND → GND.

3 Library

Tab **Library Manager** → + → LiquidCrystal
I2C.

I2C: data (SDA) and clock (SCL), plus power. The LCD's address is 0x27.

Task 3: hello, LCD

```
● ● ●
#include <Wire.h>           // I2C bus
#include <LiquidCrystal_I2C.h> // LCD over I2C

// address 0x27, 16 columns, 2 rows
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  Wire.begin(8, 9); // I2C: SDA=8, SCL=9
  lcd.init();       // start the LCD
  lcd.backlight();  // light on
  // TODO 1
}

void loop() {
  // TODO 2
}
```

Task

Open the project **LCD**.

1. **TODO 1**: show your name in **row 0**.
2. **TODO 2**: show the number of **seconds** since the start in **row 1**.

`millis()` gives the time since the start in milliseconds.

Solution: hello, LCD

```
void setup() {  
  Wire.begin(8, 9);           // I2C: SDA=8, SCL=9  
  lcd.init();                 // start the LCD  
  lcd.backlight();            // light on  
  lcd.setCursor(0, 0);        // column 0, row 0  
  lcd.print("Ksawery");       // TODO 1  
}  
  
void loop() {  
  lcd.setCursor(0, 1);        // column 0, row 1  
  lcd.print(millis() / 1000); // TODO 2: ms -> s  
  lcd.print(" s");            // the unit  
  delay(200);                 // wait 0.2 s  
}
```

Row 0

Written once in `setup()`: it never changes.

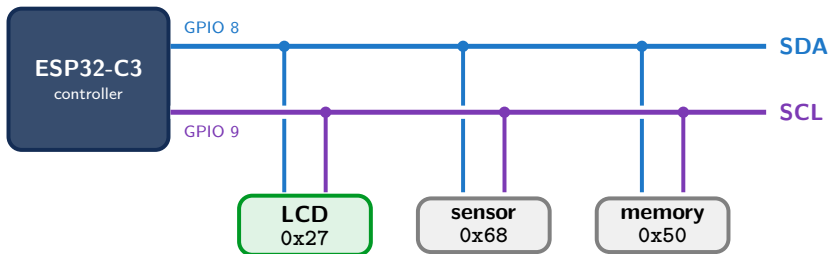
Row 1

Written again in every `loop()`, so the number keeps changing.

`delay(200)`

Not needed for the result, but the LCD does not have to be redrawn thousands of times per second.

How I2C works



1 Two wires

SCL: the clock, given by the ESP32.
SDA: one bit of data per clock tick.

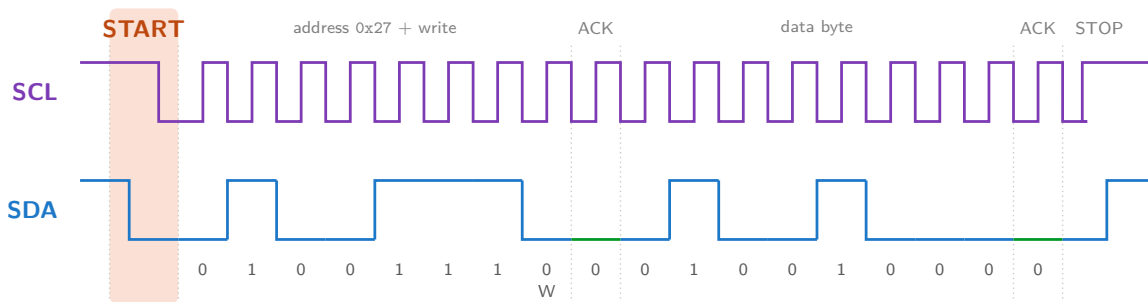
2 Addresses

Every device has its own address.
The LCD is 0x27: only it answers.

3 Messages

Address first, then the data. Next
slides: one message, step by step.

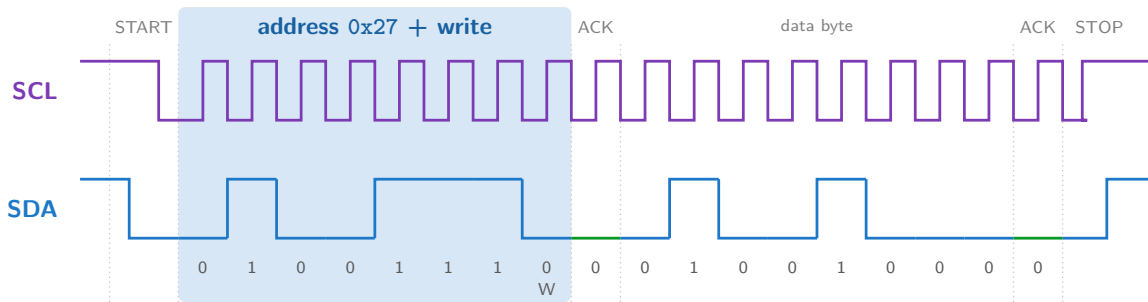
One I2C message, step by step



1 START (ESP32 → everyone)
SDA goes to 0 while SCL is 1. All devices wake up and listen: a message begins.

Rule: SDA changes only while SCL is 0. A change while SCL is 1 means START or STOP.

One I2C message, step by step

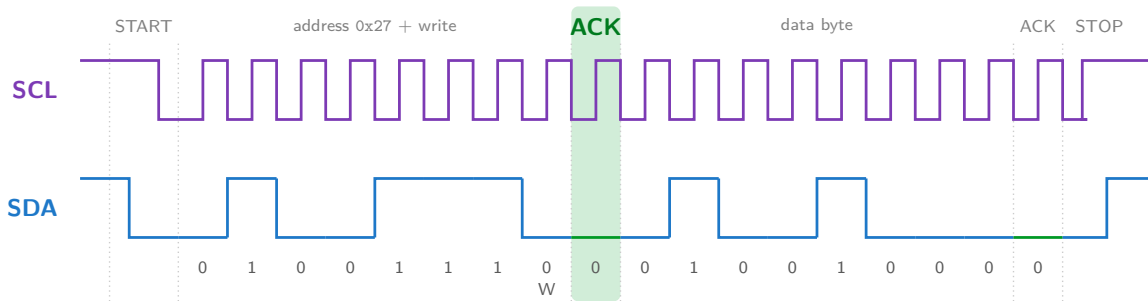


2 Address + write (ESP32 → everyone)

7 address bits: 0x27 = 0100111, one bit per SCL tick. Then 1 bit: 0 = write, 1 = read.

Rule: SDA changes only while SCL is 0. A change while SCL is 1 means START or STOP.

One I2C message, step by step

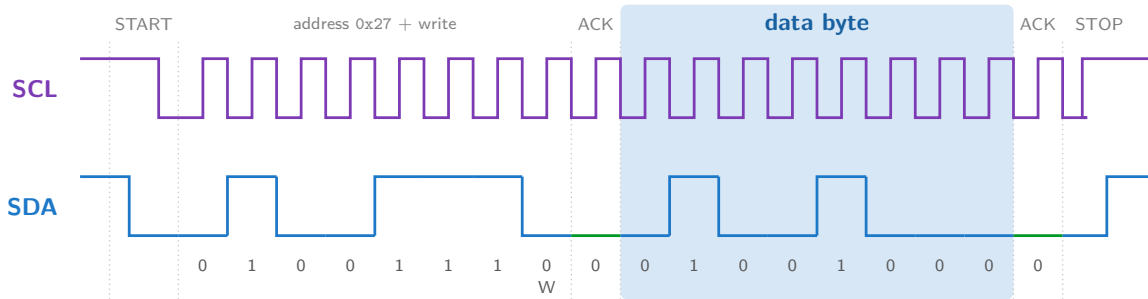


3 ACK (LCD → ESP32)

Only the device with address 0x27 answers: it pulls SDA to 0 for the 9th tick: "I am here, go on". The others stay quiet.

Rule: SDA changes only while SCL is 0. A change while SCL is 1 means START or STOP.

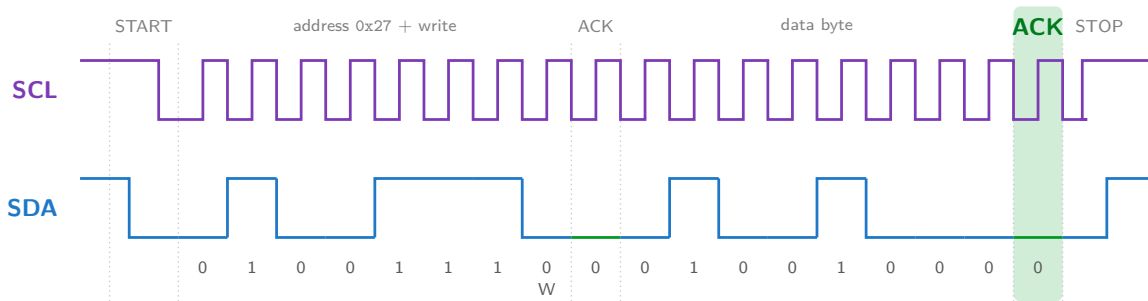
One I2C message, step by step



- 4 Data byte** (ESP32 → LCD)
8 bits of data, e.g. a command or a piece of a letter for the screen.

Rule: SDA changes only while SCL is 0. A change while SCL is 1 means START or STOP.

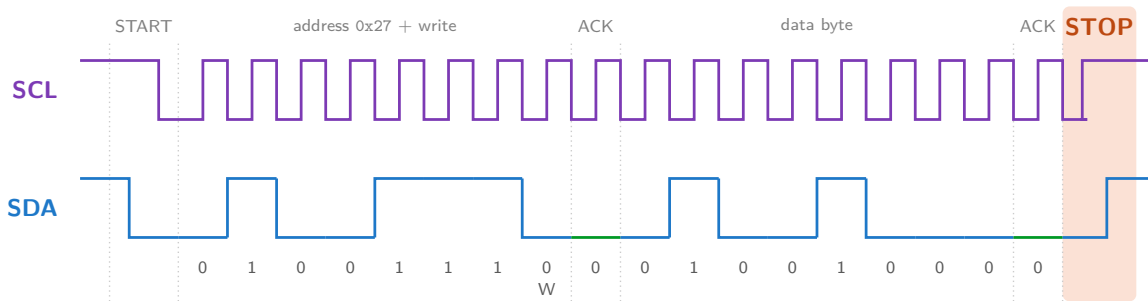
One I2C message, step by step



5 ACK (LCD → ESP32)
“Got it”. More bytes can follow, each one with its own ACK.

Rule: SDA changes only while SCL is 0. A change while SCL is 1 means START or STOP.

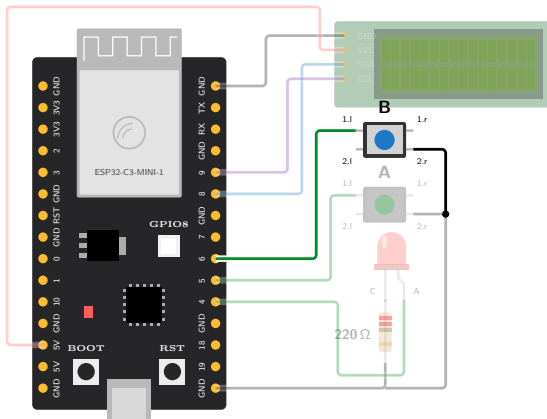
One I2C message, step by step



6 STOP (ESP32 → everyone)
SDA goes to 1 while SCL is 1: the message is over, the bus is free again.

Rule: SDA changes only while SCL is 0. A change while SCL is 1 means START or STOP.

Build it: add button B



1 Part

A second **Pushbutton**: button B.

2 Wires

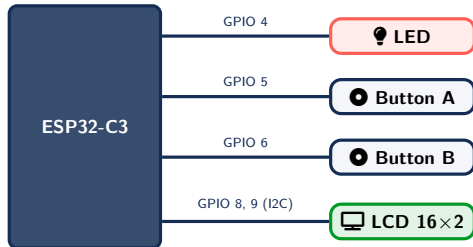
GPIO 6 → pin 1.l. Pin 2.r → **GND**. Add "bounce": "0" here too.

3 Keep it

Save the project: all later tasks use this circuit.

One GND pin can take many wires: all grounds are connected.

Task 4: our circuit for today



Task

Open the project **Circuit**. The program shows the state of both buttons.

```
● ● ●  
lcd.setCursor(0, 0);  
lcd.print("A:");  
lcd.print(digitalRead(BTN_A));  
lcd.print(" B:");  
lcd.print(digitalRead(BTN_B));
```

Run it. What does the LCD show when nothing is pressed? When you hold A? When you hold B?

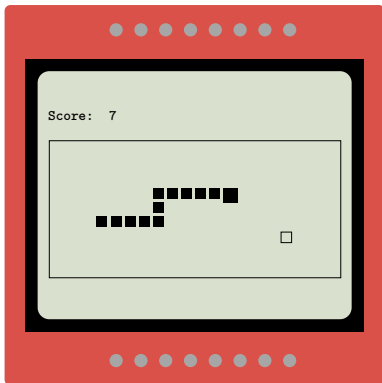
Solution: our circuit for today

Part	Pin
LED	GPIO 4
Button A	GPIO 5
Button B	GPIO 6
LCD (SDA, SCL)	GPIO 8, 9

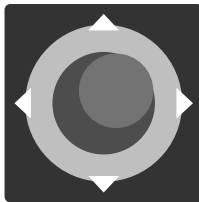
Buttons	LCD shows
nothing pressed	A:1 B:1
A held	A:0 B:1
B held	A:1 B:0

We keep this circuit for the whole day. Pressed = **0**, not pressed = **1**.

We can build real things with this



Nokia 5110: 84×48 pixels



analog joystick
2 axes + a button

🎮 Snake

The same ESP32-C3, a Nokia 5110 screen and a joystick: under 200 lines of code.

🔌 Nothing new

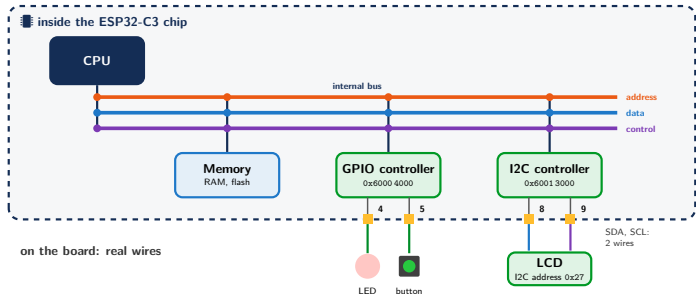
GPIO pins, a serial bus to the screen (SPI), analog inputs for the joystick, and `millis()` for the timing.

📁 Try it

The project **NokiaSnake** is on the course page.

Everything today works the same way on real hardware.

How does the CPU reach a device?



Address lines

Who: which memory cell or controller register.

Data lines

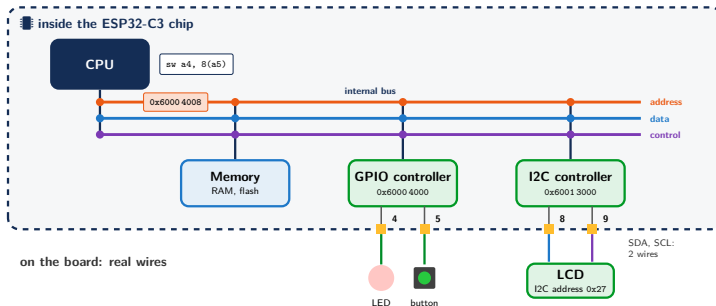
What: the value to write or the value read.

Control lines

How: read or write, and when.

The internal bus is **inside the chip**. The CPU talks only to **controllers**; they drive the pins.

One store to a device, step by step

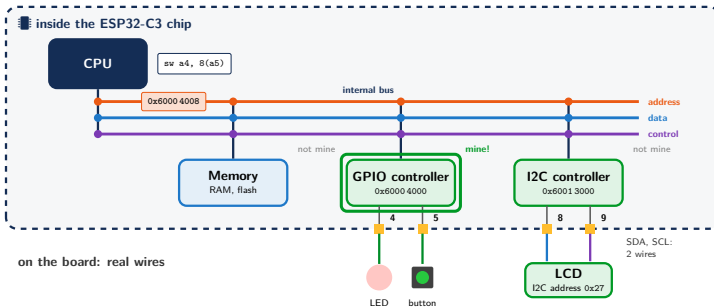


1 Address

The CPU runs `sw a4, 8(a5)` and puts the address `0x6000 4008` on the address lines. Memory and every controller see it.

The same `sw` as for memory. Only the **address** decides where the value goes.

One store to a device, step by step

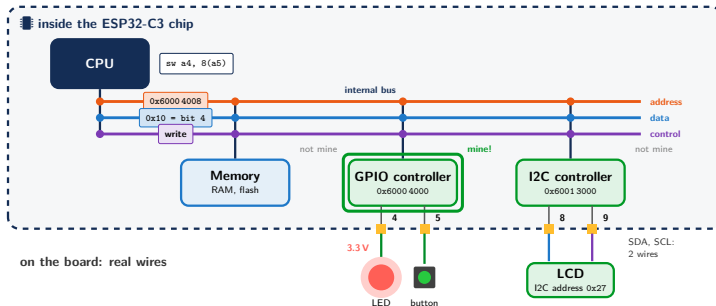


2 Who is it for?

Each one checks the address. 0x6000 4xxx belongs to the GPIO controller: *mine!* Memory and the I2C controller ignore it.

The same `sw` as for memory. Only the **address** decides where the value goes.

One store to a device, step by step

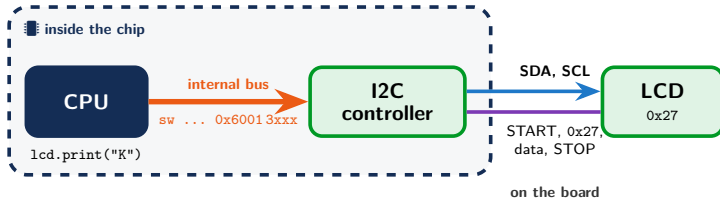


3 Data and write

The CPU puts `0x10` (bit 4) on the data lines and *write* on the control lines. The GPIO controller stores it: pin 4 goes to 3.3 V and the LED lights up.

The same `sw` as for memory. Only the **address** decides where the value goes.

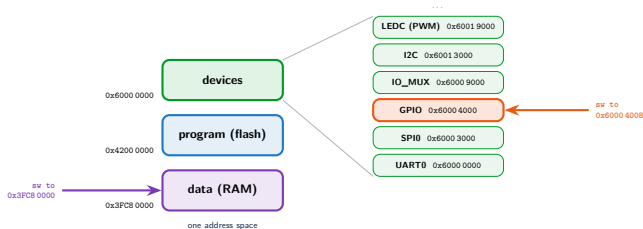
Two buses, two kinds of address



	Internal bus	I2C bus
Where	inside the chip	on the board
Wires	many, side by side	2: SDA and SCL
An address is	a register : 0x6000 4008	a device : 0x27
Speed	a transfer in 1 clock cycle	about 100 000 bits per second

`lcd.print`: the CPU writes to the I2C controller; the controller talks to the LCD over the 2 wires.

Memory-mapped I/O



Memory-mapped I/O: the registers of every controller get addresses, just like memory cells.

A store to memory

sw to 0x3FC8 0000: the value is saved in RAM.

A store to a device

sw to 0x6000 4008: the value goes to the GPIO controller.

Fixed windows

Every controller owns a range, set when the chip is designed. GPIO: 0x6000 4000–4FFF.

The same sw and lw from Ripes: the **address** decides whether it goes to memory or to a device.

The registers of the GPIO controller

Register	Address	Bit n means ...	Used by
ENABLE	0x6000 4020	1 = pin n is an output , 0 = an input	pinMode
OUT	0x6000 4004	the level to drive on pin n : 1 = 3.3 V, 0 = 0 V	
OUT_W1TS	0x6000 4008	write 1 → set bit n of OUT (0 = no change)	digitalWrite(n, HIGH)
OUT_W1TC	0x6000 400C	write 1 → clear bit n of OUT (0 = no change)	digitalWrite(n, LOW)
IN	0x6000 403C	the level that is on pin n now (read only)	digitalRead

32 bits, one per pin

In **every** register, bit n is about pin GPIO n . Bit 4 is the LED, bit 5 is button A.

Why W1TS and W1TC?

To change **one** pin with one sw. A write to OUT would change all 32 pins at once.

Task 5: an LED without digitalWrite



```
#define LED 4
#define GPIO_SET (*(volatile uint32_t *)0x60004008)
#define GPIO_CLR (*(volatile uint32_t *)0x6000400C)

void setup() {
    pinMode(LED, OUTPUT);
}

void loop() {
    GPIO_SET = 1 << LED;    // LED on
    delay(500);
    GPIO_CLR = 1 << LED;    // LED off
    delay(500);
}
```

Task

Open the project **Blink** from Task 1. Replace both digitalWrite lines with the two **yellow** lines.

0x60004008: write 1 to a bit → that pin goes **on**.

0x6000400C: write 1 to a bit → that pin goes **off**.

Does the LED blink as in Task 1?

Solution: an LED without digitalWrite

C++



```
GPIO_SET = 1 << LED;
```

What the compiler makes of it (RISC-V)



```
lui    a5, 0x60004    # a5 = 0x60004000
li     a4, 16         # 1 << 4
sw     a4, 8(a5)      # store to 0x60004008
```

Result

The LED blinks exactly as in Task 1.

What happened

One ordinary sw. Its address belongs to the GPIO device, not to RAM.

volatile

Tells the compiler: *really* do every access, this is a device.

For the CPU, a device is just a set of **addresses**.

Task 6: read the button from its register

```
● ● ●
#define GPIO_IN (*(volatile uint32_t *)0x6000403C)

void loop() {
    uint32_t reg = GPIO_IN;           // one lw
    int a = ???;                       // TODO: bit 5
    lcd.setCursor(0, 0);
    lcd.print("reg:");
    lcd.print(a);
    lcd.print(" dR:");
    lcd.print(digitalRead(BTN_A));
    delay(100);
}
```

Task

Open the project **Circuit**. The register at 0x6000403C holds the state of **all** input pins: bit n = pin n .

Complete the TODO: take bit BTN_A (= 5) of reg. Press A. Do reg and dR (digitalRead) always show the same value?

Hint: $(x \gg n) \& 1$ gives bit n of x .

Solution: read the button from its register



```
int a = (reg >> BTN_A) & 1;
```

	7	6	5	4	3	2	1	0
GPIO_IN	0	1	0	0	0	0	0	0

A pressed: bit 5 = 0

Result

reg and dR always agree: digitalRead reads the same register.

Status register

This is the **status register** of the GPIO device: one bit per pin.

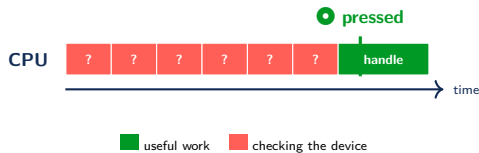
Reading a device = one 1w from its status register.

Polling: asking again and again

Polling: the CPU reads the status register again and again until something happens.

Like

Checking your mailbox every minute, all day, to see if a letter came.



Polling is simple: one loop. But what happens when the CPU also has other work?

Task 7: polling can miss events



```
int count = 0;

void loop() {
    delay(2000);                // "work": 2 s

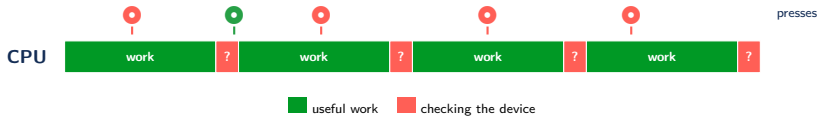
    if (digitalRead(BTN_A) == LOW) { // check once
        count++;
    }
    lcd.setCursor(0, 1);
    lcd.print(count);
}
```

Task

Open the project **Polling**. The CPU works for 2 s, then checks the button once.

1. Press button A **shortly** (click and release) **5 times** within about 10 seconds.
2. Write down the number on the LCD.

Solution: polling can miss events



Result

The LCD shows **fewer than 5**, often 0 or 1.

Why

A press counts only if the button is held **exactly** when the CPU checks. A short click during the 2 s of work is lost.

While the CPU does other work, polling **misses** events.

Task 8: polling wastes the CPU

```
uint32_t checks = 0, presses = 0;
int last = HIGH;

void loop() {
    int now = digitalRead(BTN_A);    // check
    checks++;
    if (last == HIGH && now == LOW) presses++;
    last = now;

    if (millis() >= 10000) {         // after 10 s
        lcd.setCursor(0, 0);
        lcd.print(checks);           // checks
        lcd.setCursor(0, 1);
        lcd.print(presses);          // presses
        while (true) { }             // stop
    }
}
```

Task

Open the project **Polling 2**. Now the CPU does **nothing** but check the button.

1. Press A a few times within 10 s.
2. Write down **checks** and **presses**.
3. Compute: what percentage of the checks found a new press?

Solution: polling wastes the CPU

Result

Hundreds of thousands of checks, a handful of presses.

Example: 5 presses / 400 000 checks = **0.00125%**.

In other words

More than **99.99%** of the CPU's work was wasted.

! Your numbers

Wokwi runs the CPU slower than the real chip, so you see fewer checks than a real ESP32-C3 would do. The conclusion does not change.

Polling **either** misses events (Task 7) **or** wastes the CPU (Task 8).